

AFRILANG Stdlib

std/m/listzip2

Bibliothèque AFRILANG `std/m/listzip2` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : zipAdd, zipMul, zi

```
`import "std/m/listzip2" . `use listzip2`
```

- `zipAdd(a list number, b list number) ? list number`
- `zipMul(a list number, b list number) ? list number`
- `zipSub(a list number, b list number) ? list number`
- `zipMax(a list number, b list number) ? list number`

Category: medium | Tier: medium | Functions: 4

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/m/listzip2` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : zipAdd, zipMul, zipSub, zipMax

```
`import "std/m/listzip2" . `use listzip2`  
- `zipAdd(a list number, b list number) ? list number`  
- `zipMul(a list number, b list number) ? list number`  
- `zipSub(a list number, b list number) ? list number`  
- `zipMax(a list number, b list number) ? list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/listzip2"  
use listzip2
```

Niveau : medium · Catégorie : Modules medium (m/)

Bibliothèques intermédiaires ? import std/m/?

3. Référence API

```
? zipAdd(a list number, b list number) ? list  
? zipMul(a list number, b list number) ? list  
? zipSub(a list number, b list number) ? list  
? zipMax(a list number, b list number) ? list
```

4. Exemples d'utilisation

```
import "std/m/listzip2"  
use listzip2  
  
create result = zipAdd(/* args */)   
say result  
  
create result = zipMul(/* args */)   
say result  
  
create result = zipSub(/* args */)   
say result  
  
create result = zipMax(/* args */)   
say result
```

5. Bonnes pratiques

- ? Préférez `import "std/m/listzip2"` en tête de fichier.
- ? Lancez `afrilang check` avant de livrer.
- ? Combinez avec les modules stdlib de la même catégorie.
- ? Voir /stdlib/ pour le source live et les démos playground.
- ? Signalez les problèmes sur GitHub si une export manque.

6. Annexe ? source

```
module listzip2  
  
function zipAdd(a list number, b list number) returns list number  
end  
  
function zipMul(a list number, b list number) returns list number  
end  
  
function zipSub(a list number, b list number) returns list number  
end  
  
function zipMax(a list number, b list number) returns list number  
end  
end
```

ENGLISH

1. Overview

AFRILANG library `std/m/listzip2` (tier medium, category medium). Exports 4 function(s): zipAdd, zipMul, zipSub, zipMax.

```
`import "std/m/listzip2"` . `use listzip2`  
- `zipAdd(a list number, b list number) ? list number`  
- `zipMul(a list number, b list number) ? list number`  
- `zipSub(a list number, b list number) ? list number`  
- `zipMax(a list number, b list number) ? list number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/listzip2"  
use listzip2
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries ? import std/m/?

3. API reference

```
? zipAdd(a list number, b list number) ? list  
? zipMul(a list number, b list number) ? list  
? zipSub(a list number, b list number) ? list  
? zipMax(a list number, b list number) ? list
```

4. Usage examples

```
import "std/m/listzip2"  
use listzip2  
  
create result = zipAdd(/* args */)   
say result  
  
create result = zipMul(/* args */)   
say result  
  
create result = zipSub(/* args */)   
say result  
  
create result = zipMax(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/m/listzip2"` at the top of your file.  
? Run `afriLang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module listzip2  
  
function zipAdd(a list number, b list number) returns list number  
end  
  
function zipMul(a list number, b list number) returns list number  
end  
  
function zipSub(a list number, b list number) returns list number  
end  
  
function zipMax(a list number, b list number) returns list number  
end  
end
```


